



Carrera

Analista Programador Computacional

Exp 2 – Semana 4

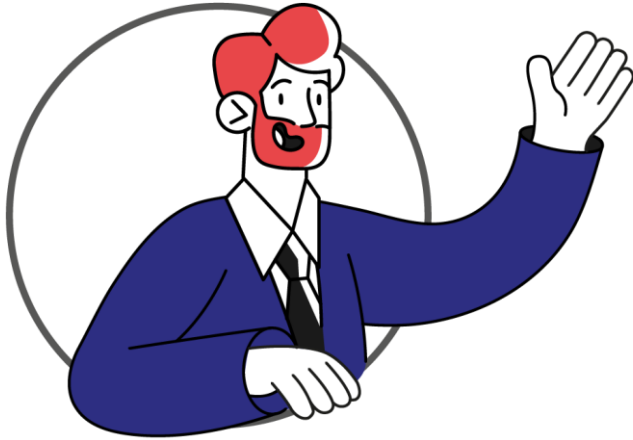
Ingeniería de Software (PRY3211)

Guía de aprendizaje

Índice

Introducción a la semana.....	3
Resultado de aprendizaje	4
El estudiante será capaz de:	4
Indicador de logro:.....	4
Conceptos relevantes	4
Preguntas activadoras	5
Actividad	5
Descripción de la actividad	5
Definiendo una visión común de una tecnología de reservas.....	6
Implementando nuestro primer modelo de datos	7
Modelamiento de Software basado en UML.....	10
Cierre de la semana	21
Referencias	22
Lecturas de la semana	22
Apunte	23

Introducción a la semana



En esta cuarta semana de nuestra experiencia 2 ya deberás tener avanzado tu proyecto por encargo y contar con el contenido declarado y aprobado de:

- Las principales Épicas y sus Historias de usuario.
- El Product Backlog con la información priorizada con sus Sprint y duración.
- El Roadmap de tu proyecto con una

proyección por semanas donde debes cumplir con sus entregables.

La primera actividad es: cumplir con la épica 0, donde debemos “Implementar un ambiente de desarrollo y producción para el sistema”.

E0-HU1 Como Scrum Team, se necesita implementar servidor de aplicaciones y de Base de datos para cargar los elementos funcionales del sistema.

E0-HU2 Como Scrum Team, necesitamos implementar frameworks de desarrollo del sistema para codificar el software.

Lo primero que debes considerar, es tener ya decididas las tecnologías de desarrollo a utilizar. Para ello, definamos un punto de vista en común sobre el Caso de proyecto:

- Definir las tecnologías de desarrollo de un Portal web o aplicación móvil que aloje nuestra información de entrada y bienvenida a los servicios del hotel.
- Definir los lenguajes de desarrollo para la aplicación de sistema que permita la gestión de la reserva.
- Definir el motor de base de datos a contener los datos del proceso de reserva.

Definiendo lo anterior, ya puedes idear cómo implementar la tecnología que permita desplegar el software de reserva.

Resultado de aprendizaje

El estudiante será capaz de:

RA2. Construye un prototipo de sistema de software basado en los estándares UML 4+1 y técnicas de prototipado.

Indicador de logro:

IL3. Utiliza los elementos del ciclo de vida para el diseño, modelado y construcción del prototipo del software desde una metodología ágil que brinde la solución requerida.

Conceptos relevantes

	UML	Casos de Uso	Vista de Proceso
	Vista Lógica	Vista de Despliegue	Diagrama de Clases
	Diagrama de actividad	Diagrama de Componentes	Modelo “4+1” de Kruchten.

Preguntas activadoras

- ¿Imaginas la relación que existe entre una pintura rupestre y una representación de un software?
- ¿Crees que es complejo explicar sólo en palabras a tu equipo de desarrolladores, al Cliente o futuro usuario, qué hace el sistema?
- ¿Te gustaría aprender una técnica UML para representar el comportamiento del sistema y sus propiedades como herramienta para definir mejor la solución a construir?

Actividad

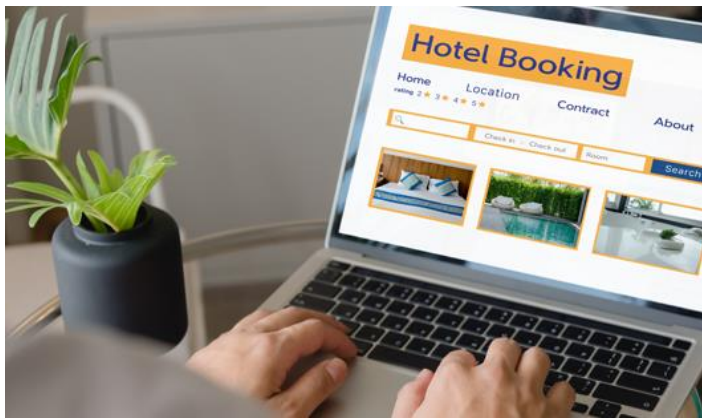
Descripción de la actividad

En esta cuarta semana realizarás una actividad formativa grupal con encargo llamada "Diseñando con técnicas UML los principales elementos del software", que aportan una propuesta de calidad para el desarrollo. Deberás modelar la vista de escenario, vistas lógicas y de procesos en UML 4+1, y además aportar la primera versión de su Base de Datos que soporte el CRUD del negocio del caso de proyecto.

Deberás transferir todos los elementos del diseño a una herramienta Git y Trello para que el equipo pueda visualizar sus avances e iterar colaborativamente, facilitando su depuración sin necesidad de estar físicamente reunidos durante el proceso.

Definiendo una visión común de una tecnología de reservas

¿Qué es una tecnología de reservas de habitaciones?



Un motor de reserva de hotel es una aplicación en línea que permite a los usuarios buscar y reservar habitaciones de hotel de manera rápida y conveniente.

Por lo tanto, lo primero que debemos definir es el modelo de datos y la base de datos implementada en nuestro ambiente de desarrollo y pruebas:

Tabla N°1:

Épica

E0-HU5	Como Scrum Team, necesitamos modelar y prototipar el producto mínimo viable de la épica 1 de reserva de habitaciones para poder determinar la visión inicial del producto y validarla con el Product-Owner y el cliente.
---------------	--

Nota. Tabla donde se muestra la épica que se debe cumplir.

Implementando nuestro primer modelo de datos



A continuación, te proporcionaremos un ejemplo de un modelo de datos básico para una reserva de habitación de hotel. Ten en cuenta que este es solo un ejemplo, y que, en la práctica, puede haber más campos y relaciones, dependiendo de los requisitos específicos del sistema.

Considera este modelo básico como la base común para comenzar a implementar nuestro principal CRUD del negocio asociado al caso.

Importante



Implementa este modelo inicial en tu servidor de base de datos para que puedas avanzar en tu proyecto.

Ya definidas las tecnologías de desarrollo de base de datos por tu equipo, haz lo siguiente:

1. Genera el código.
2. Crea la base datos.
3. Implementa el motor de base de datos.

4. Sube tus avances al control de versiones y a tu tablero ágil cargando el código y el modelo entidad relación asociado.

Luego, progresivamente en los avances y Sprint, deberás incorporar las tablas que consideres necesarias según las Historias de Usuarios que tengas declaradas en tu Product Backlog.

Tabla N°2:

Datos para cubrir principales Historias de Usuarios:

Tabla "Hotel": hotel_id (clave primaria) nombre dirección categoría	Tabla "Habitación": habitacion_id (clave primaria) hotel_id (clave foránea para relacionar con la tabla "Hotel") tipo capacidad precio
Tabla "Cliente": cliente_id (clave primaria) nombre apellido correo electrónico teléfono	Tabla "Reserva": reserva_id (clave primaria) cliente_id (clave foránea para relacionar con la tabla "Cliente") habitacion_id (clave foránea para relacionar con la tabla "Habitación") fecha_entrada fecha_salida cantidad_personas

Nota. Tabla que representa las historias de usuarios necesarios en el hotel.

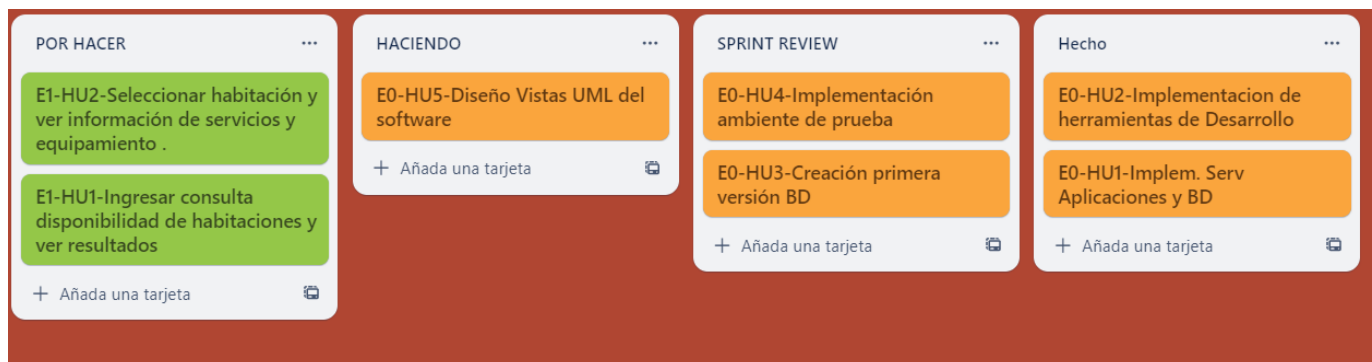
Con este modelo de datos básico, puedes realizar las siguientes operaciones:

1. Registrar hoteles con su información básica.
2. Registrar habitaciones y asociarlas con los hoteles correspondientes.
3. Registrar clientes y sus detalles de contacto.
4. Realizar reservas vinculando un cliente, una habitación y las fechas de entrada y salida.
5. Consultar información sobre hoteles, habitaciones, clientes y reservas.

Si viéramos el avance en nuestro tablero ágil a esta altura de nuestro proyecto, podríamos observar un avance de tareas como el que se observa a continuación:

Figura N°1:

Tablero ágil



Nota. Avance de tareas dentro del tablero ágil en Trello.

Importante



Recuerda que cada tarjeta debe ir avanzando en su estado de acuerdo con el progreso e instancia definida en su tablero. Cada movimiento y cambios deben verse reflejados en los comentarios de cada tarjeta y adjuntando sus evidencias y avances por cada logro.

Modelamiento de Software basado en UML

Ahora proseguiremos en las actividades que nos permitan definir el modelo del software basados en las vistas principales UML para un proyecto.

En esta etapa, reconoceremos las principales vistas recomendadas para obtener un diseño temprano de características del sistema, su comportamiento y sus principales propiedades. Para ello deberíamos cubrir las principales vistas de diseño basadas en el modelo 4+1:

Figura N°2:

Modelo “4+1” vistas de Kruchten



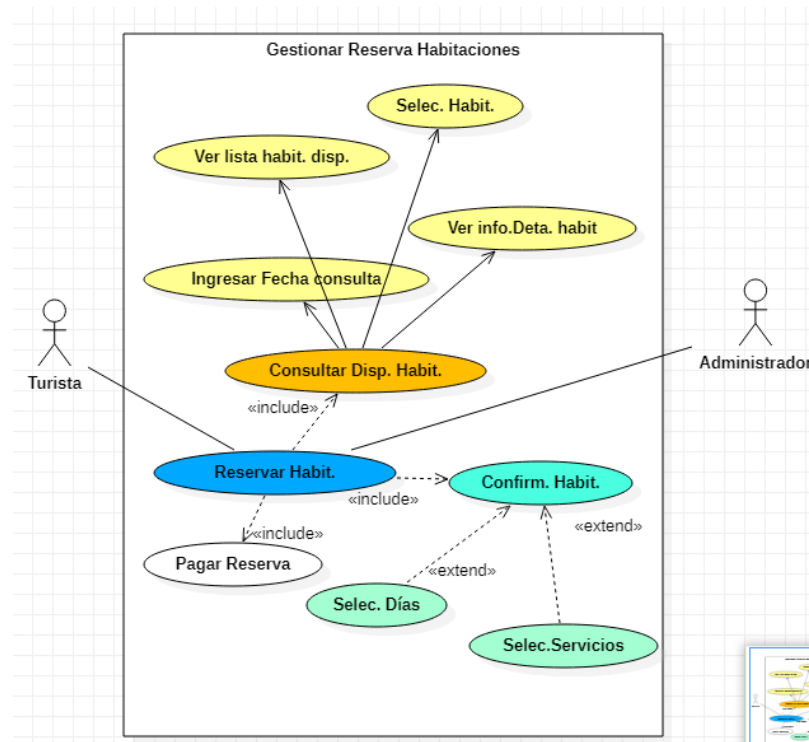
Nota. Se muestran 5 vistas principales de Kruchten: Vista de Escenarios - Vista Lógica - Vista de Despliegue - Vista Física - Vista de Procesos. Moya R. (2012). *Modelo “4+1” vistas de Kruchten (para Dummies)*. <https://jarroba.com/modelo-41-vistas-de-kruchten-para-dummies/>

Vista de escenario

Recordemos que ya nos aproximamos a la vista de escenario en un diagrama de caso de uso extendido en las experiencias anteriores y en base al caso de proyecto que estamos trabajando. Este es el modelo de entrada y es el aporte la base para definir las siguientes vistas que deberíamos representar sobre nuestra futura solución de software.

Figura N°3

Vista de escenario



Nota. Ejemplo donde se muestran los elementos de un diagrama de caso de uso extendido.

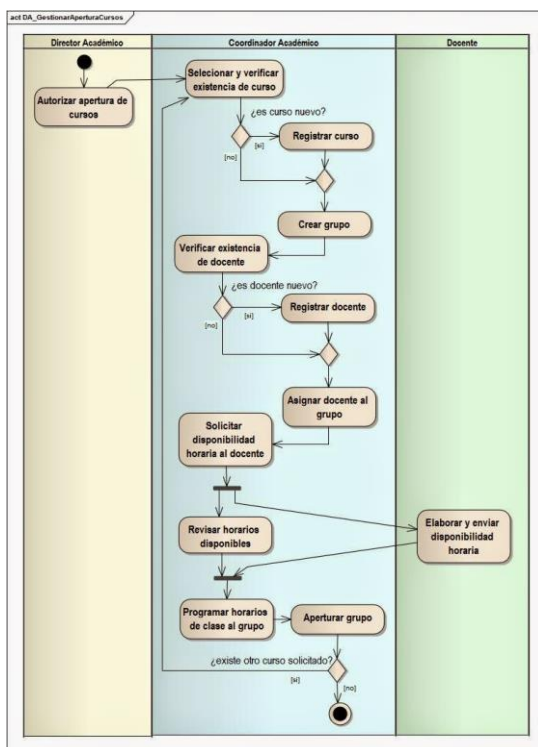
Vista de Procesos

Para la vista de procesos tenemos el “**Diagrama de actividad**” como el esquema predilecto para dar cuenta de los pasos a seguir al incorporar al sistema en los procesos definidos paso a paso.

Un diagrama de actividad es una herramienta de modelado utilizada en la ingeniería de software para visualizar el flujo de actividades en un sistema. Se utiliza para representar el comportamiento dinámico de un proceso o procedimiento, mostrando las acciones y decisiones que se toman en cada etapa.

Figura N°4

Ejemplo de diagrama de actividad.



Nota. Se muestra un diagrama de actividad con tres pasos definidos. Modelado de software. (2014). *Diagrama de actividades*. <http://modeladodesoftware.blogspot.com/2014/11/diagrama-de-actividades-caso.html>

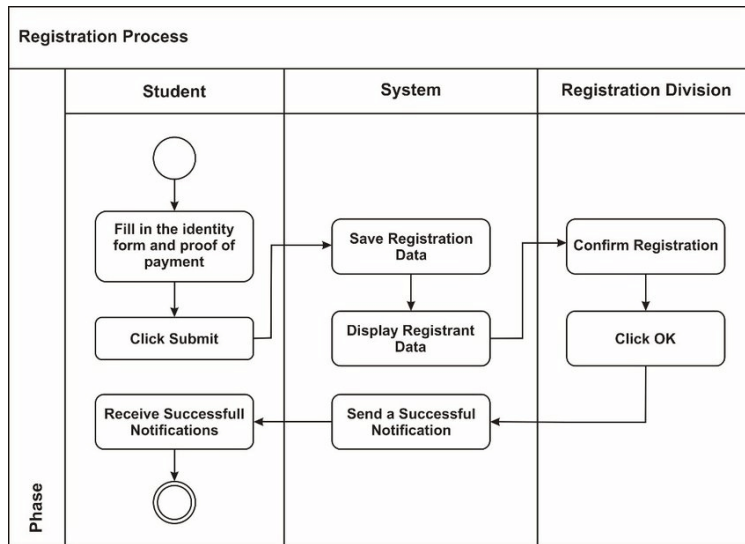
En un diagrama de actividad, las actividades se representan mediante rectángulos redondeados y se conectan mediante flechas que indican la secuencia del flujo. Además, se pueden utilizar otros elementos como rombos para representar puntos en los que se toman decisiones y bifurcaciones (barras horizontales) para mostrar múltiples caminos posibles.



Un diagrama de actividades se utiliza en conjunción del caso de uso para auxiliar a los miembros del equipo de desarrollo a entender como es utilizado el sistema y cómo reacciona en determinados eventos.

Figura N°5

Diagrama de actividades



Nota. Diagrama con “Sistema” en el workflow. Ang, J. (2022). *Cómo crear un diagrama de actividades [+Ejemplos]*. <https://es.venngage.com/blog/diagrama-de-actividades/>

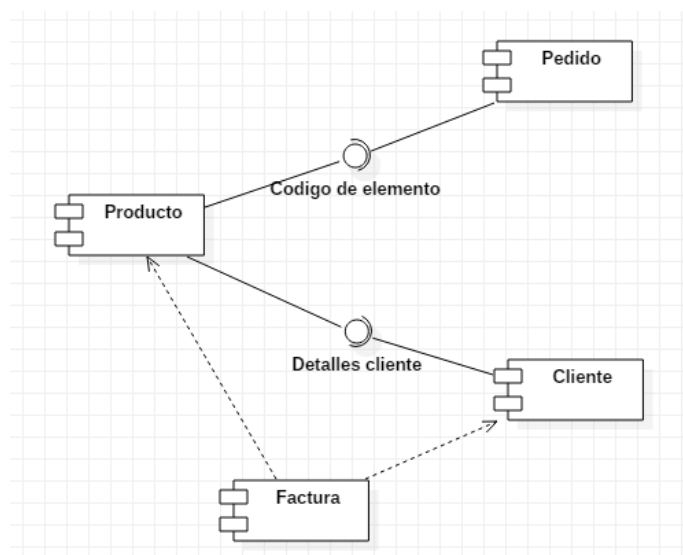
Existe la versión de diagrama que da cuenta solo de los procesos y sus actores, pero también existen versiones que incorporan al sistema en el workflow para dar cuenta de sus funciones y respuesta a las interacciones con sus usuarios. Esta sería para nosotros la mejor opción.

Vista de Despliegue:

En esta vista se muestra el sistema desde la perspectiva de un programador y se ocupa de la gestión del software; o en otras palabras, se va a mostrar cómo está dividido el sistema software en componentes y las dependencias que hay entre esos componentes. Para completar la documentación de esta vista se pueden incluir los diagramas de componentes y de paquetes de UML. El diagrama de componentes es parte de esta vista.

- **Diagrama de Componentes**

Un diagrama de componentes proporciona una visión general del sistema y documenta la organización de los componentes del sistema y sus relaciones y dependencias mutuas. El diagrama de componentes está formado por tres elementos: Componente, Interfaz y Relación de dependencia.

Figura N°6*Diagrama de componentes*

Nota. Ejemplo de un diagrama de componentes. Diagramasuml (s.f.). *Diagrama de componentes.* <https://diagramasuml.com/componentes/>

Vista Lógica

En esta vista se representa la funcionalidad que el sistema proporcionara a los usuarios finales. Es decir, se ha de representar lo que el sistema debe hacer, y las funciones y servicios que ofrece. Para completar la documentación de esta vista se pueden incluir los **diagramas de clases**, de comunicación o de secuencia de UML.

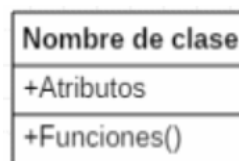
- **Diagrama de Clases**

El diagrama de clases es una herramienta visual utilizada en el desarrollo de software y en el diseño orientado a objetos para representar la estructura y las relaciones entre las clases de un sistema. Es una parte fundamental del lenguaje de modelado unificado (UML).

Muestra las clases del sistema como rectángulos con tres compartimentos principales: el superior contiene el **nombre de la clase**, el intermedio muestra los **atributos de la clase** y el inferior muestra los **métodos o funciones** asociados a la clase: Mostrar, Registrar, Eliminar, Enviar.

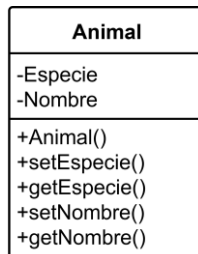
Figura N°7

Compartimentos principales de un Diagrama de clases



Nota. Notación de una clase. Diagramasuml (s.f.). *Diagrama de clases*.

<https://diagramasuml.com/diagrama-de-clases/>

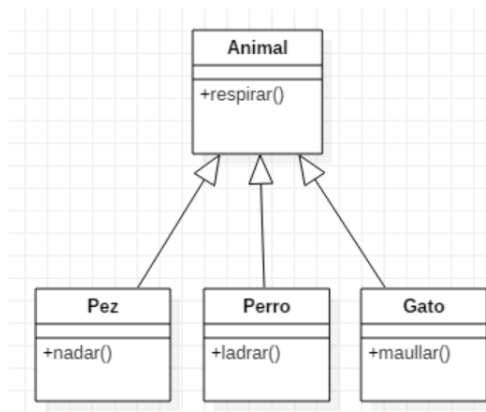
Figura N°8*Compartimentos principales de un Diagrama de clases*

Nota. Notación de una clase. Diagramasuml (s.f.). Diagrama de clases.

<https://diagramasuml.com/diagrama-de-clases/>

- **Relaciones**

Una relación identifica una dependencia. Esta dependencia puede ser entre dos o más clases (más común) o una clase hacia ella misma (menos común, pero existen), este último tipo de dependencia se denomina dependencia reflexiva. Las relaciones se representan con una línea que une las clases, esta línea variará dependiendo del tipo de relación.

Figura N°9:*Relaciones de los compartimentos especiales*

Nota. Diagramasuml. (s.f.). Diagrama de clases. <https://diagramasuml.com/diagrama-de-clases/>

Cómo dibujar un diagrama de clases:

1. Identifica los nombres de la clase: el primer paso es identificar los objetos primarios del sistema.
2. Distingue las relaciones.
3. Determina cómo cada una de las clases u objetos están relacionadas entre sí.
4. Finalmente crea la estructura de la clase.

Links de Interés



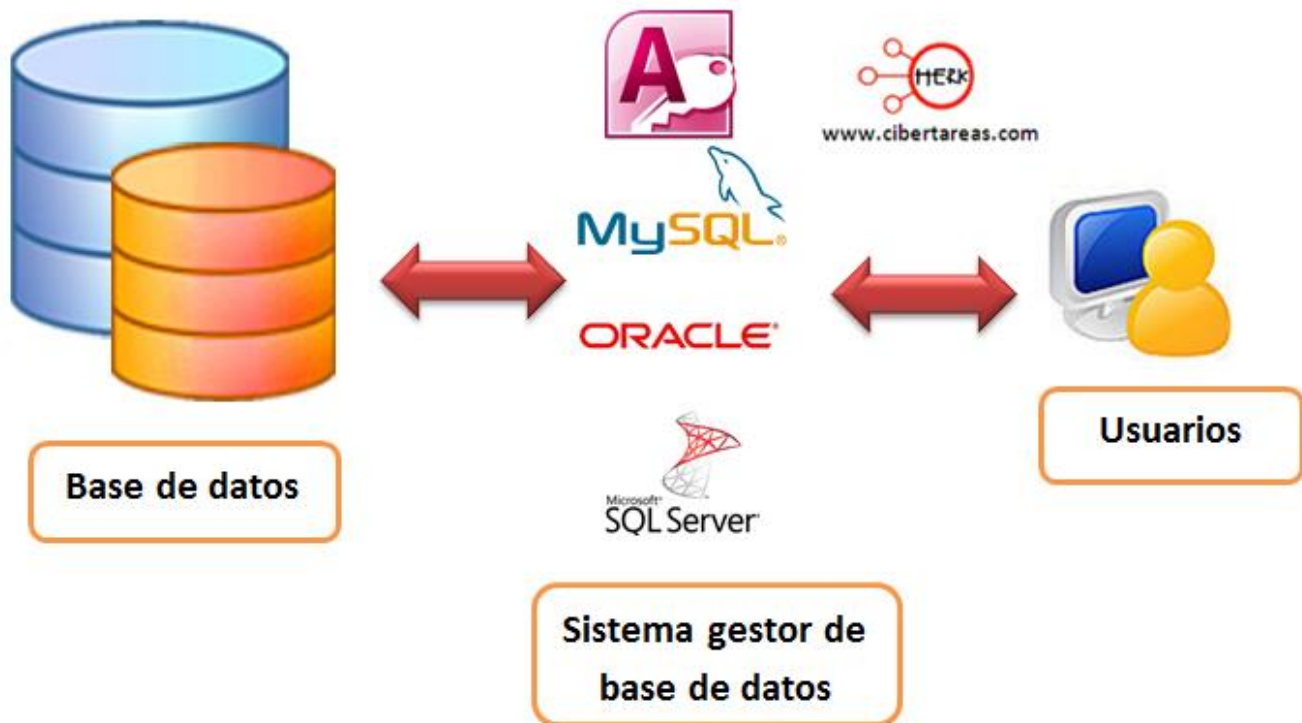
¿Quieres ver dos ejemplos de Diagrama de clases? Ingresa al siguiente link, donde verás un Sistema de Servicios bibliotecarios y un Sistema del servicio de alquiler de autos:

<https://www.webyempresas.com/ejemplos-de-diagramas-de-clases-uml/>

Vista Física

La vista física es una perspectiva o representación de un sistema de software que se enfoca en los aspectos relacionados con la implementación y la infraestructura física del sistema. Proporciona detalles sobre los componentes físicos del sistema, como hardware, servidores, redes, dispositivos de almacenamiento y otros recursos físicos necesarios para el funcionamiento del software.

Figura N°10

Vista física

Nota. Se muestran los bancos de información que contienen datos relativos. Análisis y diseño (2016). *Sistema de base de datos*. <http://johnnytite.blogspot.com/2016/01/sistema-de-base-de-datos.html>

La vista física es una parte importante de la arquitectura de software, ya que permite comprender cómo se implementa y se ejecuta el sistema en el entorno real. Complementa otras vistas, como la vista lógica o la vista de despliegue, para proporcionar una imagen completa y detallada de la arquitectura del sistema.

Links de Interés



Puedes encontrar más información en los siguientes links:

- Diagrama de casos de uso. https://videosduoc.duoc.cl/media/t/1_p8k9mr66
- Diagramas UML (s.f.). Diagrama de clases. <https://diagramasuml.com/diagrama-de-clases/>
- Modelo “4+1” vistas de Kruchten (para Dummies): <https://jarroba.com/modelo-41-vistas-de-kruchten-para-dummies/>
- Salazar, L. (2013). Historias de Usuario: un nuevo orden en los requisitos del software. <http://www.gazafatonarioit.com/2013/07/historias-de-usuario-un-nuevo-orden-en.html>

Lectura complementaria



Capítulo 6. Modelando una solución de software

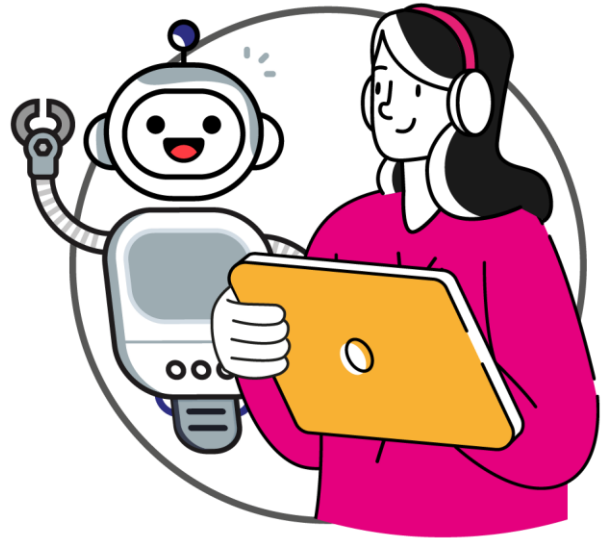
Fuente: Bravo, C. (2008). Modelando una solución de software: una visión sistémica del análisis y diseño orientado a objetos, con UML y herramientas TI. (página 299)

Cierre de la semana

El diseño de software es un proceso esencial en el desarrollo de aplicaciones, y las técnicas de Modelado de Lenguaje Unificado (UML) son ampliamente utilizadas para este propósito porque proporciona un conjunto de notaciones gráficas que permiten representar visualmente los diferentes elementos del software y las interacciones entre ellos.

Al diseñar con técnicas UML, se pueden identificar varios elementos principales del software, entre ellos: Casos de uso, Diagramas de clases, de secuencia, de actividad y de componentes.

Al utilizar técnicas UML para diseñar software, se logra una representación visual clara y comprensible de los diferentes elementos del sistema, ayudan a identificar posibles problemas de diseño y mejorar la calidad del software, permitiendo realizar ajustes antes de la implementación.



Referencias

- BettaTech-2020. Scrum y Metodologías Ágiles en Ingeniería Informática
<https://youtu.be/DT2NEBCAPHw>
- BettaTech-2021-ESTIMAR tareas de Desarrollo de la mejor forma posible
<https://youtu.be/qTz4tnGSPGg>

Lecturas de la semana

- Capítulo 6: Principios que hacen la práctica.
Fuente: Pressman, R., Maxim, B. (Comp.). (2021). Ingeniería de software: un enfoque práctico. Novena Edición. Páginas: 84-101.
- Capítulo 7: Comprensión de los requerimientos.
Fuente: Pressman, R., Maxim, B. (Comp.). (2021). Ingeniería de software: un enfoque práctico. Novena Edición. Páginas: 102-125.
- Capítulo 8: Modelado de requerimientos: un enfoque recomendado.
Fuente: Pressman, R., Maxim, B. (Comp.). (2021). Ingeniería de software: un enfoque práctico. Novena Edición. Páginas: 126 – 155.

DuocUC[®] ONLINE

Duoc UC

Facilitador disciplinar: Christian Lazcano C.

Asesor par: Ignacio Villarroel.

Reservados todos los derechos Fundación Instituto Profesional Duoc UC. No se permite copiar, reproducir, reeditar, descargar, publicar, emitir, difundir, de forma total o parcial la presente obra, ni su incorporación a un sistema informático, ni su transmisión en cualquier forma o por cualquier medio (electrónico, mecánico, fotocopia, grabación u otros) sin autorización previa y por escrito de Fundación Instituto Profesional Duoc UC. La infracción de dichos derechos puede constituir un delito contra la propiedad intelectual.